

WEB APPLICATIONS

Workshop 5: Building and Testing a REST API with JWT, OpenAPI, and Authentication

Applied to the MD Marketing & Design Project

Authors

Kevin Erazo

Steven Rodríguez

Instructor: Prof. Francisco Hidrobo

Semester: Semester II 2026

Date: October 2026

GitHub Repository

github.com/Steven735-star/APLICACIONES-WEB

Implementation, testing, and scope-decision report

Contents

1	Introduction	2
2	Objectives	2
2.1	General Objective	2
2.2	Specific Objectives	2
3	Submission Scope	2
4	Architecture	3
5	Startup and Infrastructure Verification	4
6	Workshop Requirement Mapping	5
7	Public Endpoint	6
8	User Provisioning	7
8.1	Why Public Registration Is Not Used	7
9	JWT Authentication	8
9.1	Login	8
9.2	Current Authenticated User	9
9.3	Refresh Token	10
9.4	Logout	10
9.5	Why a Separate Remember Me Mode Is Not Used	11
10	Why Social Authentication Is Not Used	11
11	Protected Customer CRUD	11
11.1	Selected Entity	11
11.2	Request Without JWT	12
11.3	Request With JWT	12
11.4	Why Individual Resource Ownership Is Not Used	13
12	OpenAPI and Swagger UI	13
13	Postman Collection Testing with Newman	14
14	Test Matrix	15
15	Security Considerations	16
16	Reduced GitHub Version	16
17	Conclusions	17

1 Introduction

Workshop 5 focuses on the implementation, protection, documentation, and testing of a REST API. The requested topics include public and protected endpoints, JWT authentication, refresh and logout mechanisms, application-specific CRUD operations, OpenAPI documentation, Swagger testing, and Postman testing.

This report applies those objectives to the real **MD Marketing & Design** project. The instructor authorized the use of the architecture that best fits each project and also allowed teams to explain why specific workshop features are not appropriate for their application. Therefore, the existing **Django + Django REST Framework + PostgreSQL** architecture is preserved instead of replacing it with FastAPI only for this workshop.

A reduced academic version is used for the submission so the repository contains only the components relevant to Workshop 5.

2 Objectives

2.1 General Objective

Demonstrate a functional REST API applied to MD Marketing & Design, protected with JWT authentication, connected to PostgreSQL, documented with OpenAPI, and tested through the required API workflows.

2.2 Specific Objectives

- Verify reproducible execution of the backend and its PostgreSQL connection.
- Demonstrate public and JWT-protected endpoints.
- Demonstrate login, access tokens, refresh tokens, and logout through token revocation.
- Provide an endpoint that returns information about the authenticated user.
- Demonstrate a complete protected CRUD over the *Customer* entity.
- Expose automatic OpenAPI documentation and Swagger UI.
- Validate successful and unsuccessful requests with Swagger, terminal commands, and Postman.
- Explain the workshop requirements that are intentionally not used because they do not fit the project's operational model.

3 Submission Scope

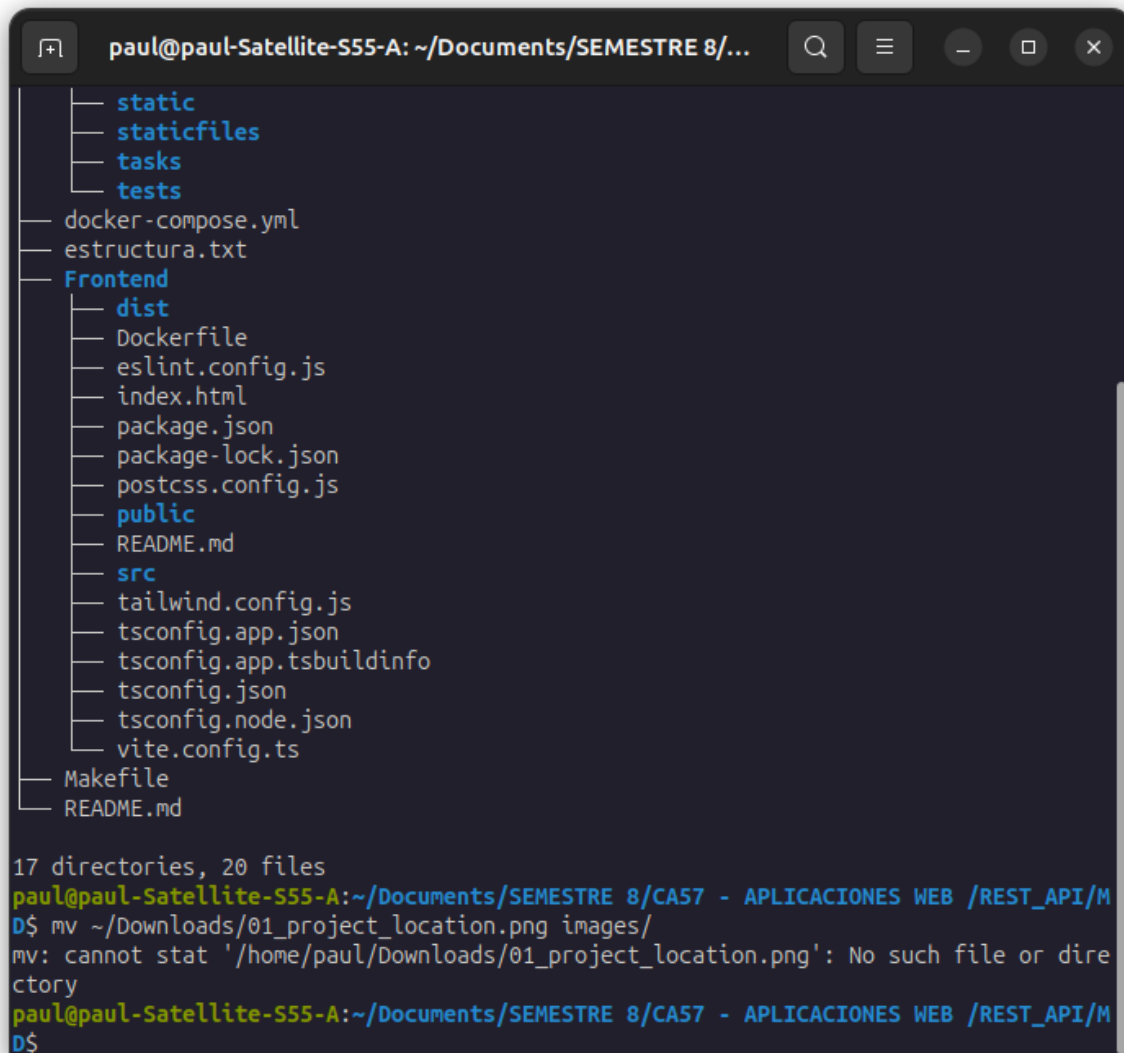
The complete MD system contains additional modules and infrastructure. For Workshop 5, the academic version intentionally isolates only the phase being evaluated.

The reduced scope includes:

- Django REST backend.
- PostgreSQL database.
- JWT authentication.
- Authenticated-user endpoint.
- Customer CRUD.
- OpenAPI and Swagger.

- Postman tests.
- Django Admin for controlled user creation.

Components such as React, Redis, Celery, Channels, inventory, finance, and production are not necessary to demonstrate this workshop phase and are therefore excluded from the reduced repository.



```
paul@paul-Satellite-S55-A: ~/Documents/SEMESTRE 8/...
├── static
│   ├── staticfiles
│   ├── tasks
│   └── tests
├── docker-compose.yml
├── estructura.txt
├── Frontend
│   ├── dist
│   ├── Dockerfile
│   ├── eslint.config.js
│   ├── index.html
│   ├── package.json
│   ├── package-lock.json
│   ├── postcss.config.js
│   ├── public
│   ├── README.md
│   ├── src
│   ├── tailwind.config.js
│   ├── tsconfig.app.json
│   ├── tsconfig.app.tsbuildinfo
│   ├── tsconfig.json
│   ├── tsconfig.node.json
│   └── vite.config.ts
├── Makefile
└── README.md

17 directories, 20 files
paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/M
D$ mv ~/Downloads/01_project_location.png images/
mv: cannot stat '/home/paul/Downloads/01_project_location.png': No such file or dire
ctory
paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/M
D$
```

Figure 1: Local Workshop 5 project directory in the development environment.

The local project path used for this workshop is:

```
~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API
```

4 Architecture

The Workshop 5 implementation uses the following technologies:

- **Django 5:** backend framework.
- **Django REST Framework:** REST API and JSON serialization.
- **SimpleJWT:** JWT access and refresh tokens.

- **PostgreSQL 16**: relational persistence.
- **drf-spectacular**: automatic OpenAPI schema and Swagger UI.
- **Docker Compose**: reproducible local execution.

The architecture can be summarized as:

Client / Swagger / Postman → Django REST API → PostgreSQL

Authentication is performed with Bearer JWT tokens for protected endpoints.

5 Startup and Infrastructure Verification

The application is started through Docker Compose:

```
docker compose up -d --build
docker compose ps
```

The backend exposes a health endpoint that verifies both the API process and the database connection:

```
curl http://localhost:8000/api/v1/health/
```

```

paul@paul-Satellite-S55-A: ~/Documents/SEMESTRE 8/...
=== WORKSHOP 5 SERVICES ===
NAME          IMAGE          COMMAND          SERVICE  CREATED      STATUS
md_db         postgres:16-alpine  "docker-entrypoint.s..."  db        4 minutes ago  Up
p 3 minutes (healthy)  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
md_web        md_marketing-web    "sh -c '\n echo 'ApL...'  web        3 minutes ago  Up
p 3 minutes (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

=== API HEALTH CHECK ===
{
  "status": "ok",
  "timestamp": "2026-10-02T18:41:29.259811+00:00",
  "database": "ok",
  "version": "1.0.0"
}
paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/MD$

```

Figure 2: Docker services running and successful API/database health verification.

6 Workshop Requirement Mapping

Table 1 explains how the Workshop 5 requirements are applied to MD Marketing & Design.

Table 1: Workshop requirements and their application in MD.

No.	Requirement	Status	Application in MD
1	Public endpoint	Implemented	GET /api/v1/health/ is accessible without JWT and verifies API/-database availability.
2	Public user registration	Not applicable	Users are created and enabled by an administrator. Public self-registration does not fit the internal identity model.
3	Username/password login	Implemented	POST /api/v1/auth/token/.
4	JWT access token	Implemented	SimpleJWT issues an access token after successful authentication.
5	Refresh token	Implemented	POST /api/v1/auth/token/refresh/.
6	Logout	Implemented	The refresh token is revoked through the token blacklist.

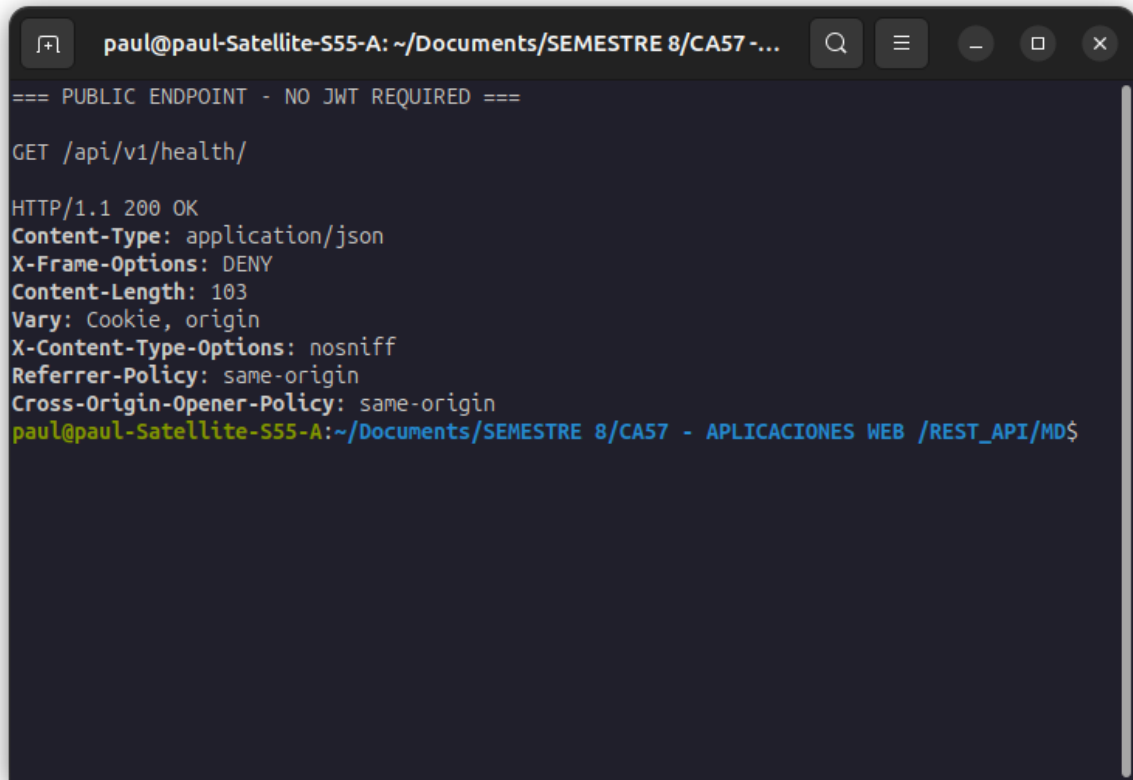
No.	Requirement	Status	Application in MD
7	Remember Me	Not applicable	Internal accounts use a uniform token-expiration policy configured by environment variables.
8	Current authenticated user	Implemented	GET /api/v1/users/me/.
9	Google/GitHub social login	Not applicable	Identities are provisioned internally by an administrator; an external identity provider is not required.
10	Application-specific entity	Implemented	The selected entity is Customer .
11	Complete CRUD	Implemented	Create, list, retrieve, update, partial update, and soft delete are available.
12	JWT-protected endpoints	Implemented	Customer operations require authenticated requests.
13	JSON requests and responses	Implemented	Django REST Framework validates and serializes JSON.
14	Automatic OpenAPI documentation	Implemented	The schema is exposed at /openapi/.
15	Swagger UI testing	Implemented	Swagger UI is exposed at /docs/.
16	Postman testing	Implemented	A Postman collection is executed with Newman to validate authentication, protected CRUD, and the current-user endpoint.

7 Public Endpoint

The health-check endpoint is intentionally public so infrastructure components can verify API and database availability without exposing sensitive business information.

```
GET /api/v1/health/
```

The endpoint returns the API status, database status, timestamp, and application version. Customers, orders, inventory, and financial data remain protected.



```
paul@paul-Satellite-S55-A: ~/Documents/SEMESTRE 8/CA57 -...
=== PUBLIC ENDPOINT - NO JWT REQUIRED ===
GET /api/v1/health/
HTTP/1.1 200 OK
Content-Type: application/json
X-Frame-Options: DENY
Content-Length: 103
Vary: Cookie, origin
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin
paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/MD$
```

Figure 3: Successful public endpoint request without a JWT.

8 User Provisioning

8.1 Why Public Registration Is Not Used

MD Marketing & Design is treated as an internal organizational application. Users are not expected to register themselves.

Instead, an authorized administrator creates and enables accounts through Django Admin. This prevents unauthorized account creation and matches the operational model of the system.

The absence of a public `/auth/register` endpoint is therefore a deliberate security and identity-governance decision.

The administrator interface is available at:

```
http://localhost:8000/admin/
```

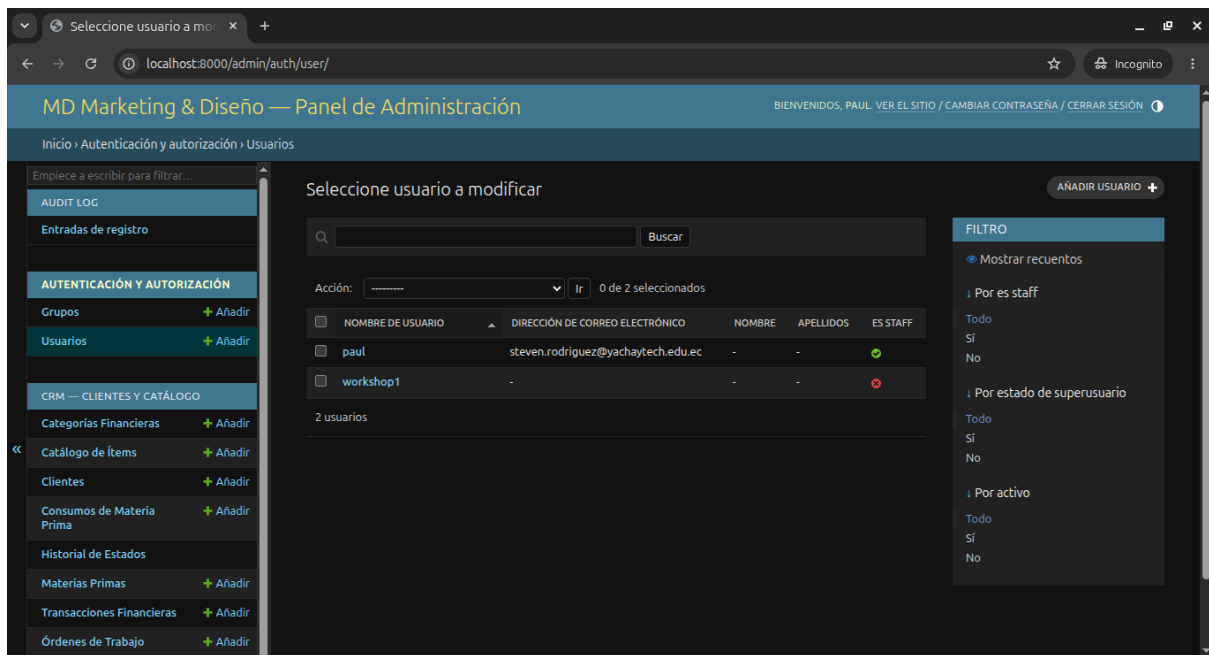


Figure 4: Administrator-controlled user creation in Django Admin.

9 JWT Authentication

9.1 Login

The login endpoint receives Django-managed credentials and returns an access token and a refresh token.

```
POST /api/v1/auth/token/  
Content-Type: application/json  
  
{  
  "username": "admin_w5",  
  "password": "*****"  
}
```

A successful response contains both JWT values.

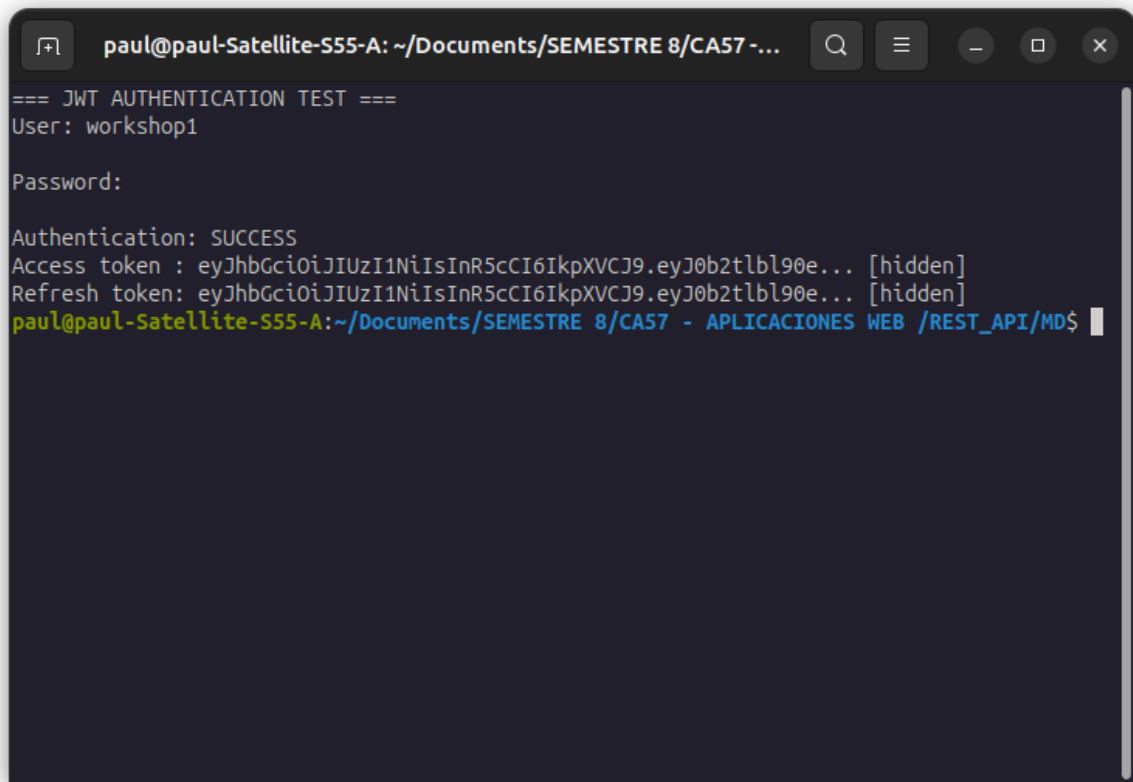
A terminal window with a dark background. The title bar shows the user 'paul@paul-Satellite-S55-A' and the current directory '~/Documents/SEMESTRE 8/CA57 -...'. The terminal output shows a 'JWT AUTHENTICATION TEST' where the user 'workshop1' provides a password. The authentication is successful, and two JWT tokens are generated: an access token and a refresh token, both displayed as long alphanumeric strings followed by '[hidden]'. The prompt at the bottom is 'paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/MD\$'.

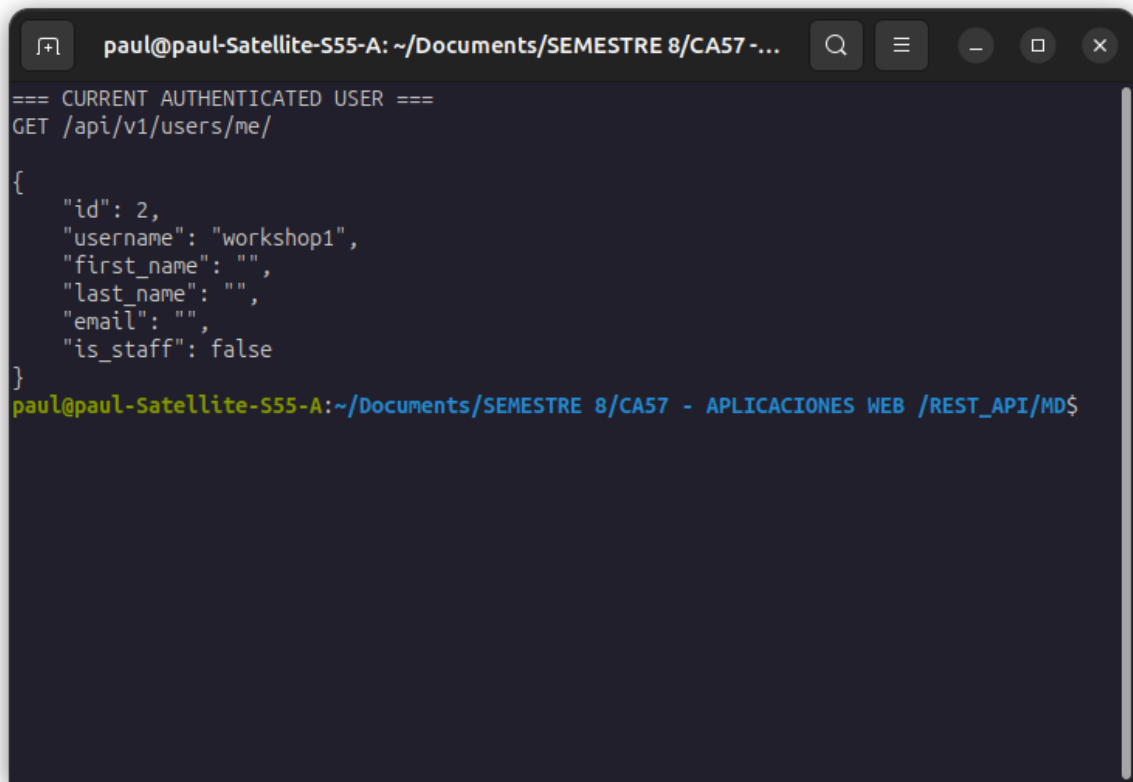
Figure 5: Successful login and generation of JWT access and refresh tokens.

9.2 Current Authenticated User

The authenticated-user endpoint requires a valid access token:

```
GET /api/v1/users/me/  
Authorization: Bearer <access-token>
```

The response includes safe account information such as the user ID, username, name, email, and staff status. It does not expose the password or password hash.



```
paul@paul-Satellite-S55-A: ~/Documents/SEMESTRE 8/CA57 -...
=== CURRENT AUTHENTICATED USER ===
GET /api/v1/users/me/

{
  "id": 2,
  "username": "workshop1",
  "first_name": "",
  "last_name": "",
  "email": "",
  "is_staff": false
}
paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/MD$
```

Figure 6: Authenticated request to the current-user endpoint.

9.3 Refresh Token

A valid refresh token can be exchanged for a new access token:

```
POST /api/v1/auth/token/refresh/
Content-Type: application/json

{
  "refresh": "<refresh-token>"
}
```

The configuration enables refresh-token rotation and blacklisting of previously used refresh tokens.

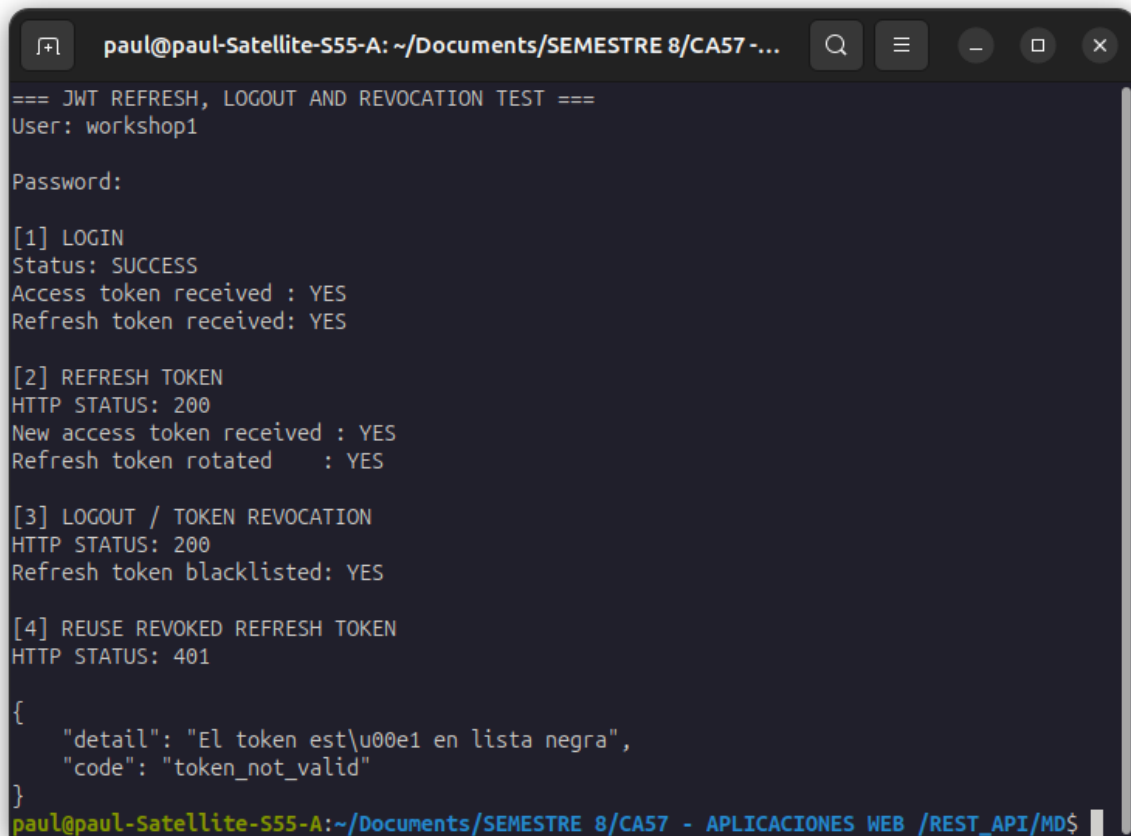
9.4 Logout

Logout is implemented by sending the current refresh token to the blacklist endpoint exposed by the Workshop 5 API:

```
POST /api/v1/auth/token/blacklist/
Content-Type: application/json

{
  "refresh": "<refresh-token>"
}
```

After revocation, the refresh token cannot be reused.



```

paul@paul-Satellite-S55-A: ~/Documents/SEMESTRE 8/CA57 -...
=== JWT REFRESH, LOGOUT AND REVOCATION TEST ===
User: workshop1

Password:

[1] LOGIN
Status: SUCCESS
Access token received : YES
Refresh token received: YES

[2] REFRESH TOKEN
HTTP STATUS: 200
New access token received : YES
Refresh token rotated    : YES

[3] LOGOUT / TOKEN REVOCATION
HTTP STATUS: 200
Refresh token blacklisted: YES

[4] REUSE REVOKED REFRESH TOKEN
HTTP STATUS: 401

{
  "detail": "El token est\u00e1 en lista negra",
  "code": "token_not_valid"
}
paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/MD$

```

Figure 7: Refresh-token renewal and logout/revocation workflow.

9.5 Why a Separate Remember Me Mode Is Not Used

The project uses a uniform JWT expiration policy for internal accounts. Token lifetimes are centrally configured through environment variables.

A separate user-selectable Remember Me mode is not required by the current operational workflow. The access token remains temporary and the refresh-token lifetime is centrally controlled.

10 Why Social Authentication Is Not Used

Google or GitHub authentication is not integrated in the Workshop 5 version.

The system uses administrator-managed institutional accounts. Allowing an external social identity provider would introduce a second account-provisioning mechanism that does not correspond to the current business process.

For this reason, social login is documented as intentionally out of scope rather than as a missing technical capability.

11 Protected Customer CRUD

11.1 Selected Entity

The **Customer** entity belongs to the real MD domain and is used to demonstrate the protected CRUD requirements.

The available routes are:

```

POST /api/v1/crm/clientes/
GET /api/v1/crm/clientes/

```

```
GET /api/v1/crm/clientes/{id}/  
PUT /api/v1/crm/clientes/{id}/  
PATCH /api/v1/crm/clientes/{id}/  
DELETE /api/v1/crm/clientes/{id}/
```

The DELETE operation is a soft delete: the customer is marked inactive instead of being physically removed from the database.

11.2 Request Without JWT

Customer operations require authentication. A request without an access token must be rejected.

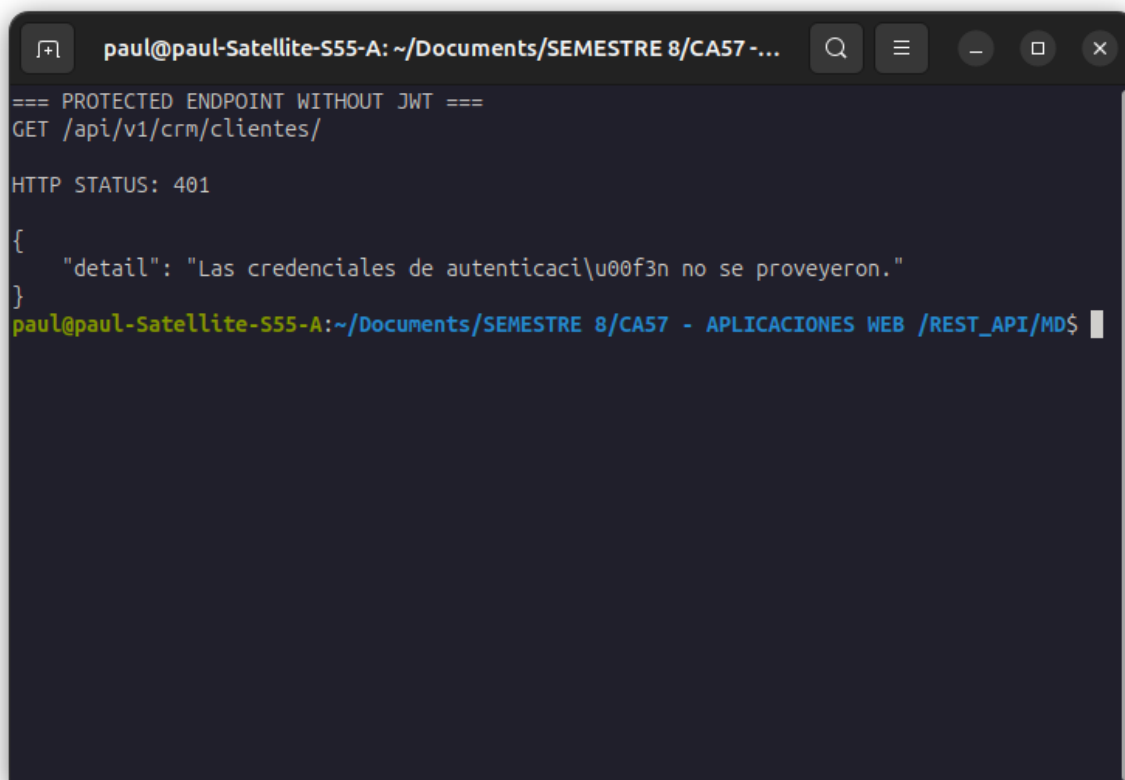


Figure 8: Protected customer endpoint rejected when no JWT is provided.

11.3 Request With JWT

After successful login, the access token is sent using the standard Bearer authorization header.

```
Authorization: Bearer <access-token>
```

The authenticated user can then execute the permitted customer operations.

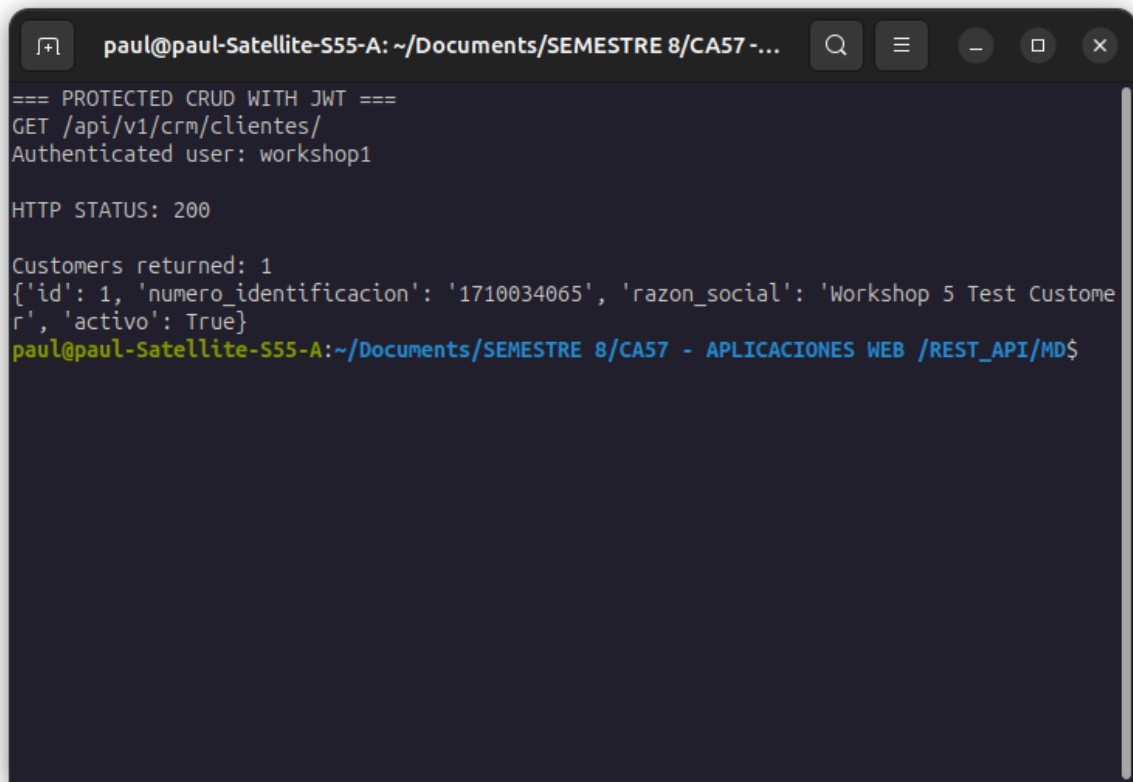
A terminal window with a dark background and light-colored text. The window title is 'paul@paul-Satellite-S55-A: ~/Documents/SEMESTRE 8/CA57 -...'. The terminal output shows a REST client request: 'GET /api/v1/crm/clientes/' with the header 'Authenticated user: workshop1'. The response is 'HTTP STATUS: 200' and 'Customers returned: 1', followed by a JSON object: '{\'id\': 1, \'numero_identificacion\': \'1710034065\', \'razon_social\': \'Workshop 5 Test Customer\', \'activo\': True}'. The prompt at the bottom is 'paul@paul-Satellite-S55-A:~/Documents/SEMESTRE 8/CA57 - APLICACIONES WEB /REST_API/MD\$'.

Figure 9: Successful protected customer CRUD request using a Bearer JWT.

11.4 Why Individual Resource Ownership Is Not Used

The workshop discusses scenarios in which a resource belongs to the user who created it. That model does not directly fit MD customers.

Customers are organizational resources shared by authorized staff, not personal objects belonging to an individual account. Therefore, the application does not enforce a rule such as:

```
customer.user_id == current_user.id
```

Authentication is required for the CRUD. If the business process later requires finer authorization, role-based permissions can be added for areas such as administration, sales, production, or finance.

12 OpenAPI and Swagger UI

The Workshop 5 API uses `drf-spectacular` to generate its OpenAPI schema.

The documentation endpoints are:

```
GET /openapi/  
GET /docs/
```

Swagger UI allows the team to inspect operations, schemas, request bodies, response models, status codes, and authentication requirements.

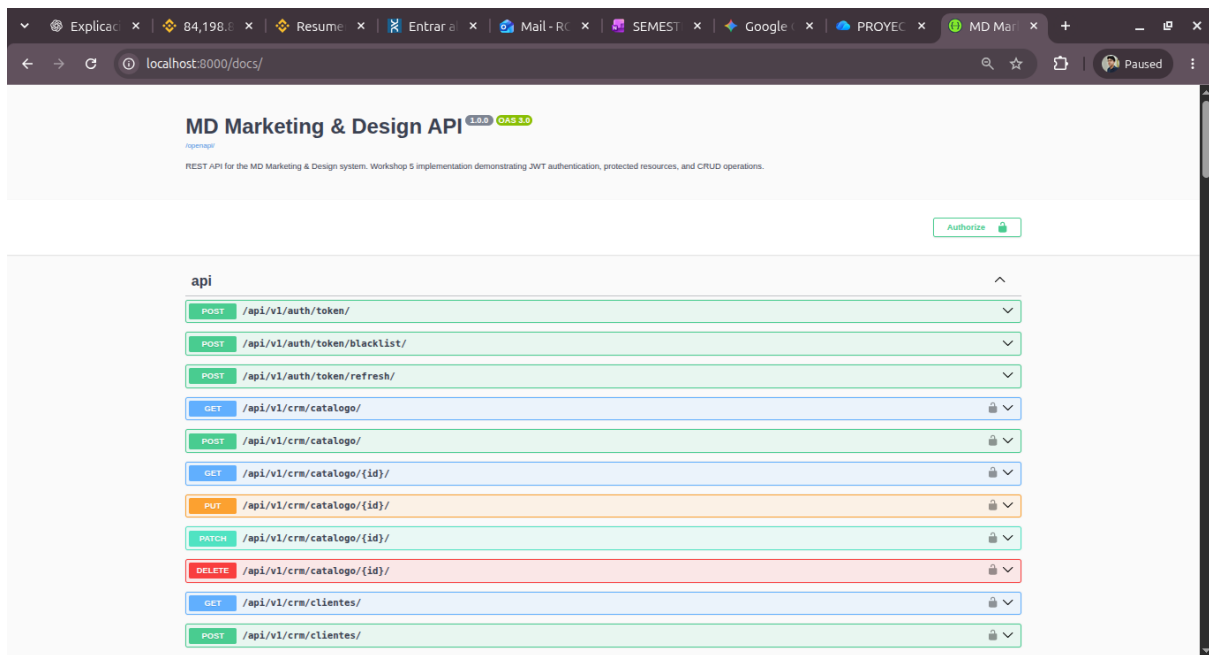


Figure 10: Swagger UI displaying the Workshop 5 API endpoints and schemas.

13 Postman Collection Testing with Newman

A Postman collection is used to reproduce the complete authentication and CRUD workflow independently of the browser-based Swagger interface. The collection is executed with Newman, Postman's command-line collection runner, because the desktop client was unstable in the local Ubuntu environment.

The test workflow includes:

- public endpoint without authentication;
- correct and incorrect login attempts;
- authenticated `/users/me/`;
- protected CRUD without JWT;
- protected CRUD with JWT;
- valid and invalid refresh token requests;
- logout and refresh-token revocation.

```

=== POSTMAN COLLECTION TEST WITH NEWMAN ===

Password for workshop1:

newman

MD Marketing & Design - Workshop 5

→ 1 - Login
  POST http://127.0.0.1:8000/api/v1/auth/token/ [200 OK, 750B, 928ms]
  ✓ Login returns HTTP 200
  ✓ Access token returned
  ✓ Refresh token returned

→ 2 - Get Customers - JWT Protected
  GET http://127.0.0.1:8000/api/v1/crm/clientes/ [200 OK, 543B, 243ms]
  ✓ Protected endpoint returns HTTP 200

→ 3 - Users Me
  GET http://127.0.0.1:8000/api/v1/users/me/ [200 OK, 355B, 192ms]



|            | executed | failed |
|------------|----------|--------|
| iterations | 1        | 0      |
| requests   | 3        | 0      |


```

Figure 11: Postman collection executed with Newman: three requests and four assertions completed with zero failures.

14 Test Matrix

Test Case	Expected Result	Treatment in MD
Public endpoint without JWT	Success	Implemented
Valid public registration	User created	Not applicable : administrator-controlled provisioning
Duplicate/invalid registration data	Rejected	Not applicable : no public registration
Correct login	Tokens returned	Implemented
Incorrect login	401 Unauthorized	Implemented
/users/me/ without token	Rejected	Implemented
/users/me/ with valid token	User returned	Implemented
Protected CRUD without JWT	Rejected	Implemented
Protected CRUD with JWT	Success	Implemented
Access another user's resource	Rejected	Adapted : customers are organizational resources rather than user-owned records
Invalid or expired JWT	Rejected	Implemented
Valid refresh token	New access token	Implemented
Invalid refresh token	Rejected	Implemented
Logout	Refresh token revoked	Implemented

Test Case	Expected Result	Treatment in MD
Reuse revoked refresh token	Rejected	Implemented
Remember Me false/true	Different refresh life-time	Not applicable : uniform internal token policy
Social registration/login	User authenticated	Not applicable : internal administrator-managed identities
Swagger/OpenAPI	Success	Implemented
Postman/Newman workflow	Success	Implemented

15 Security Considerations

The Workshop 5 version applies the following controls:

- Passwords are managed by Django’s password-hashing system and are never stored as plain text.
- API responses do not return password hashes.
- Database credentials and application secrets are loaded from environment variables.
- The repository distributes `.env.example`, not the real `.env`.
- Business endpoints require JWT authentication.
- Access and refresh tokens are distinguished.
- Refresh tokens can be rotated and revoked through the blacklist mechanism.
- Customer request data is validated through Django REST Framework serializers.
- Customer deletion is logical, preserving record traceability.

Complete JWT values, passwords, and other secrets must be hidden or cropped in screenshots used in the report or GitHub repository.

16 Reduced GitHub Version

For the workshop submission, a reduced repository is used instead of publishing the complete MD application.

Its conceptual structure is:

```
REST_API/
|-- Backend/
| |-- apps/
| | '-- crm/
| |-- config/
| |-- Dockerfile
| |-- manage.py
| '-- requirements.txt
|-- postman/
| '-- MD_Workshop5.postman_collection.json
|-- .env.example
|-- .gitignore
|-- docker-compose.yml
|-- README.md
'-- main.tex
```

The Workshop 5 project is available in the following GitHub repository:

<https://github.com/Steven735-star/APLICACIONES-WEB>

17 Conclusions

The Workshop 5 phase demonstrates that MD Marketing & Design provides a REST API protected by JWT, connected to PostgreSQL, documented with OpenAPI, and applied to a real domain CRUD.

The implementation preserves the architecture of the original project rather than replacing it solely to match an example technology. Requirements that do not fit the real operational model are explicitly justified.

In particular, public self-registration, social login, a separate Remember Me mode, and individual customer ownership are not used because MD is modeled as an internal organizational system with administrator-managed accounts and shared business resources.

The reduced repository keeps the submission focused on the API, authentication, documentation, and testing concerns evaluated in Workshop 5.